

COURSE MATERIAL

SUBJECT	BASIC ELECTRONICS ENGINEERING (EE23AES101)
UNIT	3
COURSE	B.TECH
DEPARTMENT	ECE
SEMESTER	1-1
PREPARED BY (Faculty Name/s)	P.SRINIVASULU Assistant Professor
Version	V-1
PREPARED / REVISED DATE	22-11-2023

TABLE OF CONTENTS – UNIT 1

S.No.	CONTENTS	PAGE No.
1	COURSE OBJECTIVES	1
2	PREREQUISITES	1
3	SYLLABUS	1
4	COURSE OUTCOMES	1
5	CO - PO/PSO MAPPING	2
6	LESSON PLAN	2
7	ACTIVITY BASED LEARNING	3
8	LECTURE NOTES	
	3.1 Number System	4
	3.2 Binary Codes	12
	3.3 Digital logic operations and Gates	17
	3.4 Boolean Algebra	21
	3.5 Boolean functions	25
	3.6 Two level NAND & NOR implementations of Boolean functions	26
	3.7 Combinational Circuits	30
	3.8 Sequential Circuits	32
9	PRACTICE QUIZ	38
10	ASSIGNMENTS	41
11	PART A QUESTIONS & ANSWERS (2 MARKS QUESTIONS)	42
12	PART B QUESTIONS	43
13	SUPPORTIVE ONLINE CERTIFICATION COURSES	44
14	REAL TIME APPLICATIONS	44
15	CONTENTS BEYOND THE SYLLABUS	45
16	PRESCRIBED TEXT BOOKS & REFERENCE BOOKS	52
17	MINI PROJECT SUGGESTION	53

1. Course Objectives

The objectives of this course is to

- To familiarize with the concepts of different number systems and Boolean algebra.
- To understand the design techniques of combinational, sequential logic circuits.
- To outline procedures for the analysis and design of combinational and sequential logic circuits.

2. Prerequisites

Students should have knowledge on

- Basic mathematical operations

3. Syllabus

Unit 3

Overview of Number Systems, Logic gates including Universal Gates, BCD codes, Excess-3 code, Gray code, Hamming code. Boolean Algebra, Basic Theorems and properties of Boolean Algebra, Truth Tables and Functionality of Logic Gates – NOT, OR, AND, NOR, NAND, XOR and XNOR. Simple combinational circuits–Half and Full Adder, Introduction to sequential circuits, Flip flops, Registers and counters (Elementary Treatment only)

Course outcomes

After completion of the course, student will be able to

CO1: Understand the properties of Boolean algebra, other logic operations.

CO2: Make use of the concepts to solve the problems related to the logic Gates.

CO3: Analyze the combinational and sequential logic circuits.

4. CO-PO / PSO Mapping

BEE	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	P10	PO11	PO12	PSO1	PSO2
CO1	3	2	3	2	2	2							3	3
CO2	3	2	3	3	2	2							3	3
CO3	3	3	3	3	3	2							3	3

5. Lesson Plan

Lecture No.	Weeks	Topics to be covered	References
1	1	Number System – Binary, Octal, Decimal and Hexadecimal - Introduction, Conversion of numbers from decimal to other number systems	T1
2		Conversion of numbers from other number systems to decimal, Conversion of Octal and hexadecimal numbers to and from binary	T1
3		Binary Codes – BCD, Excess 3, Gray code	T1
4		Complements	T1
5	2	Logic Gates and operations	T1
6		Boolean Algebra- Properties and basic theorems	T1
7		Boolean Algebra- Properties and basic theorems	T1
8		Complements of Boolean functions, Realization of basic gates using NAND and NOR gates	T1
9	3	Combinational circuits–Half and Full Adder	T1
10		SR-FLIP-FLOP, JK-FLIP-FLOP, T-FLIP-FLOP, D-FLIP-FLOP,	T1
11		Shift Register using D-Flip flop	T1
12		Design of counters using JK flip flop	T1

7. Activity Based Learning

1. Practical verification of functionality of different types of logic gates and implementing Boolean functions, etc.
2. Technical Quiz.
3. Problem solving activity in Just A Minute.

8. Lecture Notes

3.1 NUMBER SYSTEM

The numbers are necessary for quantitative analysis. The number systems which are commonly used in the applications implemented using digital systems are:

- Binary Number System
- Octal Number System
- Decimal Number System
- Hexa Decimal Number System

All the above specified number systems are positional number systems, i.e., the value of the digit (place value) depends on its position in the number.

The most familiar number system for human beings is the decimal number system, but this system is not convenient for digital system which operates on only two values (on ('1') or off ('0')).

The base or radix of a number system ('r') is defined as the number of distinct symbols (digits or characters) used in that number system. The range of values for a digit in a number system with base 'r' is from 0 to (r-1).

For example, the Base '5' number system has 5 digits whose value ranges from 0 to 4.

Number System	Radix ('r')	Digits or characters available in that number system
Binary	2	"0", "1"
Octal	8	"0", "1", "2", "3", "4", "5", "6", "7"
Decimal	10	"0", "1", "2", "3", "4", "5", "6", "7", "8", "9"
Hexa Decimal	16	"0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "A", "B", "C", "D", "E", and "F". Where "A" stands for 10, "B" for 11 and so on.

NUMBER BASE CONVERSIONS

In general, a number with a decimal point (or radix point) is represented by a series of coefficients:

$$a_n \dots a_2 a_1 a_0 . a_{-1} a_{-2} a_{-3} \dots a_{-m}$$

In general, a number expressed in a base-r system has coefficients multiplied by powers of r:

$$a_n X r^n + a_{n-1} X r^{n-1} + \dots + a_2 X r^2 + a_1 X r^1 + a_0 + a_{-1} X r^{-1} + a_{-2} X r^{-2} + \dots + a_{-m} X r^{-m}$$

Where the value of coefficient a_j ranges from 0 to $r - 1$.

For example, $(7392)_{10}$ is written as $7 \times 10^3 + 3 \times 10^2 + 9 \times 10^1 + 2 \times 10^0$.

To distinguish between numbers of different bases, we enclose the coefficients in parentheses and write a subscript equal to the base used.

Conversion of numbers in any base to Decimal

The conversion of a number in base r to decimal is done by expanding the number in a power series and adding all the terms. The conversion from any base to base 10 can be obtained using the formulae

$$(N)_{10} = \sum_{j=-m}^{n-1} a_j r^j$$

Where r is the base, a_j is the digit at position j , m is the number of digits after the decimal point, n is the number of digits of the integer part and $(N)_{10}$ is the obtained number in decimal. Representations of a number in a different radix are said to be equivalent if they have the same decimal representation.

For example, $(0011)_8 = (1001)_2 = (9)_{10}$.

Conversion of numbers from Binary to Decimal

1. Convert $(110011)_2$ to decimal.

$$\begin{aligned}(110011)_2 &= 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 1 \times 32 + 1 \times 16 + 0 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 \\ &= 32 + 16 + 0 + 0 + 2 + 1 = (51)_{10}\end{aligned}$$

$$(110011)_2 = (51)_{10}$$

2. Convert $(11010.11)_2$ to decimal.

$$\begin{aligned}(11010.11)_2 &= 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} \\ &= 1 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 + 1 \times 0.5 + 1 \times 0.25 \\ &= 16 + 8 + 0 + 2 + 0 + 0.5 + 0.25 = (26.75)_{10}\end{aligned}$$

$$(11010.11)_2 = (26.75)_{10}$$

3. Convert $(1010.101)_2$ to decimal.

$$\begin{aligned}(1010.101)_2 &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\ &= 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 + 1 \times 0.5 + 0 \times 0.25 + 1 \times 0.125 \\ &= 8 + 0 + 2 + 0 + 0.5 + 0.125 = (10.625)_{10}\end{aligned}$$

$$(1010.101)_2 = (10.625)_{10}$$

Conversion of numbers from Octal to Decimal

1. Convert $(314)_8$ to decimal.

$$\begin{aligned}(314)_8 &= 3 \times 8^2 + 1 \times 8^1 + 4 \times 8^0 \\ &= 3 \times 64 + 1 \times 8 + 4 \times 1\end{aligned}$$

$$= 192 + 8 + 4 = (204)_{10}$$

$$(314)_8 = (204)_{10}$$

2. Convert $(12.56)_8$ to decimal.

$$(12.56)_8 = 1 \times 8^1 + 2 \times 8^0 + 5 \times 8^{-1} + 6 \times 8^{-2}$$

$$= 1 \times 8 + 2 \times 1 + 5 \times 0.125 + 6 \times 0.015625$$

$$= 8 + 2 + 0.625 + 0.09375 = (10.71875)_{10}$$

$$(12.56)_8 = (10.71875)_{10}$$

Conversion of numbers from Hexa Decimal to Decimal

1. Convert $(BF401)_{16}$ to decimal.

$$(BF401)_{16} = B \times 16^4 + F \times 16^3 + 4 \times 16^2 + 0 \times 16^1 + 1 \times 16^0$$

$$= 11 \times 65536 + 15 \times 4096 + 4 \times 256 + 0 \times 16 + 1 \times 1$$

$$= 720896 + 61440 + 1024 + 0 + 1 = (783361)_{10}$$

$$(BF401)_{16} = (783361)_{10}$$

2. Convert $(432.EC)_{16}$ to decimal.

$$(432.EC)_{16} = 4 \times 16^2 + 3 \times 16^1 + 2 \times 16^0 + E \times 16^{-1} + C \times 16^{-2}$$

$$= 4 \times 256 + 3 \times 16 + 2 \times 1 + 14 \times 0.0625 + 12 \times 0.00390625$$

$$= 1024 + 48 + 2 + 0.875 + 0.046875 = (1074.921875)_{10}$$

$$(432.EC)_{16} = (1074.921875)_{10}$$

Conversion of numbers from other bases to Decimal

1. Convert $(231.5)_5$ to decimal.

$$(231.5)_5 = 2 \times 5^2 + 3 \times 5^1 + 1 \times 5^0 + 5 \times 5^{-1}$$

$$= 2 \times 25 + 3 \times 5 + 1 \times 1 + 5 \times 0.2$$

$$= 50 + 15 + 1 + 1 = (67)_{10}$$

$$(231.5)_5 = (67)_{10}$$

2. Convert $(A12.B)_{12}$ to decimal.

$$(A12.B)_{12} = A \times 12^2 + 1 \times 12^1 + 2 \times 12^0 + B \times 12^{-1}$$

$$= 10 \times 144 + 1 \times 12 + 2 \times 1 + 11 \times 0.08333$$

$$= 1440 + 12 + 2 + 0.91667 = (1454.91667)_{10}$$

$$(A12.B)_{12} = (1454.91667)_{10}$$

Conversion of numbers in decimal to any base

- The conversion of a decimal integer to a number in base r is done by dividing the number and all successive quotients by r and accumulating the remainders.

- The conversion of a decimal *fraction* to a number in base r is done by multiplying that fraction and all successive fractions by r and accumulating the integers. The process is continued until the fraction becomes 0 or until the number of digits has sufficient accuracy.
- The conversion of decimal numbers with both integer and fractional parts is done by converting the integer and the fraction separately and then combining the two answers.

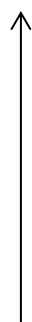
This procedure is best illustrated by the following examples.

Decimal to Binary Conversion

1. $(41)_{10} = (?)_2$

Representation I or

41		
20	-	1
10	-	0
5	-	0
2	-	1
1	-	0
0	-	1



Successive division of
number and its quotients

by base value

$$41/2$$

$$20/2$$

$$10/2$$

$$5/2$$

$$2/2$$

$$1/2$$

Quotients Remainders

$$20 \quad 1$$

$$10 \quad 0$$

$$5 \quad 0$$

$$2 \quad 1$$

$$1 \quad 0$$

$$0 \quad 1$$

Stop the process when quotient becomes zero. By accumulating the remainders from bottom to top we get the equivalent binary number of the given decimal number.

$$(41)_{10} = (101001)_2$$

2. $(0.685)_{10} = (?)_2$

Successive
multiplication of
fractions by base value

Fractional
Part

Integer Part

$$0.685 \times 2 = 1.37$$

$$0.37$$

$$1$$

$$0.685 \times 2 = 1.37$$

$$0.74$$

$$0$$

$$0.74 \times 2 = 1.48$$

$$0.48$$

$$1$$

$$0.48 \times 2 = 0.96$$

$$0.96$$

$$0$$

$$0.96 \times 2 = 1.92$$

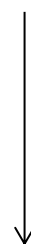
$$0.92$$

$$1$$

$$0.92 \times 2 = 1.84$$

$$0.84$$

$$1$$



The process is continued until the fraction becomes 0 or until the number of digits has sufficient accuracy. By accumulating the integer values only from top to bottom we get the equivalent binary number of the given decimal fraction.

$$(0.685)_{10} = (0.101011.....)_2$$

3. $(49.75)_{10} = (?)_2$

As the given decimal number has both integer and fractional part, there is a need to separate the integer and fractional part and each part is to be converted separately. Finally, the obtained results are to be combined.

49					
24	-	1			
12	-	0			
6	-	0			
3	-	0			
1	-	1			
0	-	1			

Successive multiplication of fractions by base value

$0.75 \times 2 = 1.5$

$0.5 \times 2 = 1.0$

Fractional Part

0.5

0.0

Integer Part

1

1

$(49)_{10} = (110001)_2$

$(0.75)_{10} = (0.11)_2$

$(49.75)_{10} = (110001.11)_2$

Decimal to Octal Conversion

1. $(143)_{10} = (?)_8$

14				
17	-	7		
2	-	1		
0	-	2		

Successive division of number and its quotients by base value

$143/8$

$17/8$

$2/8$

Quotients

17

2

0

Remainders

7

1

2

$$(143)_{10} = (217)_8$$

2. $(0.7685)_{10} = (?)_8$

Successive
multiplication of
fractions by base
value

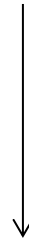
Fractional
Part

Integer Part

$0.7685 \times 8 = 6.148$
 $0.148 \times 8 = 1.184$
 $0.184 \times 8 = 1.472$
 $0.472 \times 8 = 3.776$
 $0.776 \times 8 = 6.208$
 $0.208 \times 8 = 1.664$
 $\vdots$
 $\vdots$

0.148
 0.184
 0.472
 0.776
 0.208
 0.664
 $\vdots$
 $\vdots$

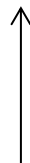
6
 1
 1
 3
 6
 1
 $\vdots$
 $\vdots$



$(0.7685)_{10} = (0.611361.....)_8$

3. $(18.76)_{10} = (?)_{16}$

1	18	
1	1	- 2
	0	- 1



Successive
multiplication of
fractions by base value

Fractional
Part

Integer
Part

$0.76 \times 16 = 12.16$
 $0.16 \times 16 = 0.256$
 $0.56 \times 16 = 8.96$
 $\vdots$
 $\vdots$

0.16
 2.56
 0.96
 $\vdots$
 $\vdots$

$12 (C)$
 2
 8
 $\vdots$
 $\vdots$



$(18)_{10} = (12)_8$

$(0.76)_{10} = (0.C28)_{16}$

$(18.76)_{10} = (12.C28....)_{16}$

Octal and Hexadecimal Number Conversions

The conversion from and to binary, octal, and hexadecimal plays an important role in digital computers, because shorter patterns of hex characters are easier to recognize than long patterns of 1's and 0's. Since $2^3=8$ and $2^4=16$, each octal digit corresponds to three binary digits and each hexadecimal digit corresponds to four binary digits.

Binary to Octal Conversion:

Numbers in Binary can be converted to octal conversion by partitioning the binary number into groups of three digits each, starting from the binary point and proceeding to the left and to the right. If there are less than 3 bits in a group pad with zeros. The corresponding octal digit is then assigned to each group.

$$\begin{array}{c} \longleftrightarrow \\ (10110001101011 . 111100000110)_2 = (26153.7406)_8 \\ 2 \quad 6 \quad 1 \quad 5 \quad 3 \quad . \quad 7 \quad 4 \quad 0 \quad 6 \end{array}$$

$$\begin{array}{c} \longleftrightarrow \\ (001101011 . 111100)_2 = (153.74)_8 \\ 1 \quad 5 \quad 3 \quad . \quad 7 \quad 4 \quad 7 \quad 4 \end{array}$$

Binary to Hexadecimal Conversion:

Conversion from binary to hexadecimal is similar to that of binary to octal, except that the binary number is divided into groups of *four* digits:

$$\begin{array}{c} \longleftrightarrow \\ (010001101011 . 111100000110)_2 = (46B.F06)_{16} \\ 4 \quad 6 \quad B \quad . \quad F \quad 0 \quad 6 \end{array}$$

Octal or Hexadecimal to Binary Conversion:

Octal or hexadecimal to binary conversion is done by reversing the preceding process. Each octal digit is converted to its three-digit binary equivalent also known as Binary Coded Octal. Similarly, each hexadecimal digit is converted to its four-digit binary equivalent also known as Binary Coded Hexadecimal.

$$\begin{array}{c} \longleftrightarrow \\ (673.124)_8 = (110 \ 111 \ 011 . 001 \ 010 \ 100)_2 \\ 6 \quad 7 \quad 3 \quad . \quad 1 \quad 2 \quad 4 \end{array}$$

$$\begin{array}{c} \longleftrightarrow \\ (306.D)_{16} = (0011 \ 0000 \ 0110 . 1101)_2 \\ 3 \quad 0 \quad 6 \quad . \quad D \end{array}$$

During communication between people (about binary numbers in the computer), the octal or hexadecimal representation is more desirable because it can be expressed more compactly with a third or a quarter of the number of digits required for the equivalent binary number. Thus, most computer manuals use either octal or hexadecimal numbers to specify binary quantities.

Numbers with different bases

Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
00	0000	00	00
01	0001	01	01
02	0010	02	02
03	0011	03	03
04	0100	04	04
05	0101	05	05
06	0110	06	06
07	0111	07	07
08	1000	10	08
09	1001	11	09
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

3.2. BINARY CODES

Digital system represents and manipulates not only binary numbers, but also many other discrete elements of information. Any discrete element of information that is distinct among a group of quantities can be represented with a binary code (a pattern of 0's and 1's).

An n-bit binary code is a group of n-bits that assumes up to 2^n distinct combinations of 1's and 0's with each combination representing one element of the set that is being coded. The bit combination of an n-bit code is determined from the count in binary from 0 to (2^n-1) . Although the minimum number of bits required to code 2^n distinct quantities is n, there is no limit on the maximum number of bits may be used for a binary code, for example 10 decimal digits can be coded with 10 bits also.

Binary codes are mainly classified into 5 groups –

- Weighted binary codes
- Non-weighted binary codes
- Error detecting codes
- Error correcting codes
- Alpha Numeric codes

Based on the properties, binary codes can be classified as –

- Sequential Codes
- Self-Complementing / Reflective Codes

Weighted Binary Codes:

In a weighted binary code, each bit position is assigned a weighting factor in such a way that each digit can be evaluated by adding the weights of all the 1's in the coded combination. The commonly used weighted codes are-

- 8 4 2 1 Code (Binary Coded Decimal)
- 2 4 2 1 Code
- 5 4 2 1 Code
- 5 2 1 1 Code
- 8 4 -2 -1 Code

For example, In 8 4 2 1 code the value of 0110 is evaluated as

$$8 \times 0 + 4 \times 1 + 2 \times 1 + 1 \times 0 = 4 + 2 = 6.$$

In 2 4 2 1 code 1101 can be evaluated as

$$2 \times 1 + 4 \times 1 + 2 \times 0 + 1 \times 1 = 2 + 4 + 1 = 7.$$

Note: Some digits can be coded in two possible ways in the 2 4 2 1 code. (For example, the digit 4 can be assigned with 0100 or 1010).

Binary Coded Decimal

Even though the most natural system for a computer is binary number system, people are more accustomed to the decimal system. So, a decimal number is need to be converted to binary, and then after processing using the digital systems it is to be converted to decimal.

- The code most commonly used for the decimal digits is the straight binary assignment, which is known as Binary Coded Decimal (BCD).
- The BCD is weighted code with weights 8 4 2 1.

- The 10 decimal digits form a set, which required minimum 4 bits to code a single digit. Using 4 bits 16 bit combinations can be obtained, out of which 6 combinations are unused.
- The binary numbers 0000 to 1001 are similar to BCD, but 1010 to 1111 are unused in BCD, and have no meaning in BCD.
- A number with k decimal digits require 4k bits in BCD representation.
- Even though, BCD representation occupies more space than a pure binary representation, it has an advantage that the input and output data are generated by people who use decimal system.

Decimal Symbol	Binary Number	BCD digit 8 4 2 1
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
8	1000	1000
9	1001	1001
10	1010	0001 0000
11	1011	0001 0001
12	1100	0001 0010
13	1101	0001 0011
14	1110	0001 0100
15	1111	0001 0101

Ex: $(395)_{10} = (0011\ 1001\ 0101)_{BCD}$

$(97)_{10} = (1001\ 0111)_{BCD}$

$(35)_8 = (29)_{10} = (0010\ 1001)_{BCD}$

Other weighted codes:

Decimal Symbol	BCD 8 4 2 1	5 4 2 1	2 4 2 1	5 2 1 1	8 4 -2 -1
0	0000	0000	0000	0000	0000
1	0001	0001	0001	0001	0111
2	0010	0010	0010	0011	0110
3	0011	0011	0011	0101	0101
4	0100	0100	0100	0111	0100
5	0101	1000	1011	1000	1011
6	0110	1001	1100	1010	1010
7	0111	1010	1101	1100	1001
8	1000	1011	1110	1110	1000
9	1001	1100	1111	1111	1111

In the above specified weighted codes, out of 16 combinations only 10 combinations are used, remaining bit combinations are unused.

Non -weighted Binary Codes:

In non-weighted binary codes, no weightage is assigned to bit positions. The commonly used non-weighted codes are-

- Excess 3 Code
- Gray Code

Excess 3 Code:

Excess 3 is a non-weighted code. An excess 3 code of a decimal digit can be obtained by adding 3 to the decimal number and then representing in binary. It is also self-complementing or reflective code.

Ex: To obtain excess 3 code for 6, 3 is added to 6 ($6 + 3 = 9$) and then represented in its equivalent binary (1001).

$(395)_{10}$ in excess 3 code can be represented as 0110 1100 1000

$(97)_{10}$ in excess 3 code can be represented as 1100 1010

$(35)_8$ in excess 3 code can be represented as $(29)_{10} = 0101 1100$

Decimal Symbol	BCD digit 8 4 2 1	Excess 3 code
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Gray Code:

The gray code is a non-weighted code and it has advantage over straight binary number sequence is that, successive code words differ by only one bit from one number to the next. Hence it is also called Unit Distance Code. Since gray code is a non-weighted code it is seldom used in arithmetic applications, but it is used in analog to digital converters. It is an example for cyclic code.

Decimal Symbol	Binary Number	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111

11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Conversion of binary numbers to gray code:

Let binary number sequence of $n+1$ bits represented as $b_n b_{n-1} b_{n-2} \dots b_2 b_1 b_0$.

The gray code of the above specified binary sequence can be obtained as

$$b_n (b_n \oplus b_{n-1}) (b_{n-1} \oplus b_{n-2}) \dots (b_2 \oplus b_1) (b_1 \oplus b_0)$$

Where $\oplus$ is XOR operator, it results one if two bits are unequal, otherwise zero.

For example, the gray code for 1110 is obtained as $1(1 \oplus 1)(1 \oplus 1)(1 \oplus 0) = 1001$

The gray code for 0101 is obtained as $0(0 \oplus 1)(1 \oplus 0)(0 \oplus 1) = 0111$

Conversion of gray code to binary number:

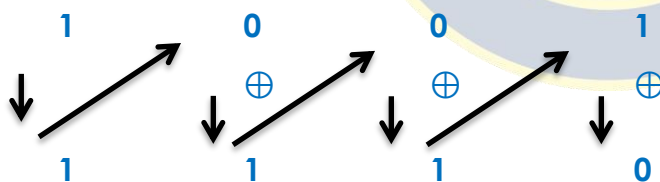
Let gray code of $n+1$ bits represented as $g_n g_{n-1} g_{n-2} \dots g_2 g_1 g_0$.

The binary sequence $b_n b_{n-1} b_{n-2} \dots b_2 b_1 b_0$ of the above specified gray code can be obtained as $g_n (b_n \oplus g_{n-1}) (b_{n-1} \oplus g_{n-2}) \dots (b_2 \oplus g_1) (b_1 \oplus g_0)$

Where $g_n = b_n, b_{n-1} = (b_n \oplus g_{n-1}) \dots$ and so on.

For example, the binary sequence for gray code 1001 is obtained as

$$1(1 \oplus 0)(1 \oplus 0)(1 \oplus 1) = 1110.$$



Sequential Codes

In Sequential codes each binary code is one binary number greater than its preceding code. The 8,4,2,1 code, excess-3 code are examples for sequential codes.

3.3. DIGITAL LOGIC OPERATIONS AND GATES

Logic Gate is a digital circuit which performs some logic functions. The logic gates can be broadly classified in three categories:

- Basic Gates – AND, OR, NOT

- Universal Gates – NAND, NOR
- Exclusive Gates - XOR, XNOR

Factors to be weighed in considering the construction of other types of logic gates are:

- (1) The feasibility and economy of producing the gate with physical components,
- (2) The possibility of extending the gate to more than two inputs,
- (3) The basic properties of the binary operator, such as commutativity and associativity, and
- (4) The ability of the gate to implement Boolean functions alone or in conjunction with other gates.

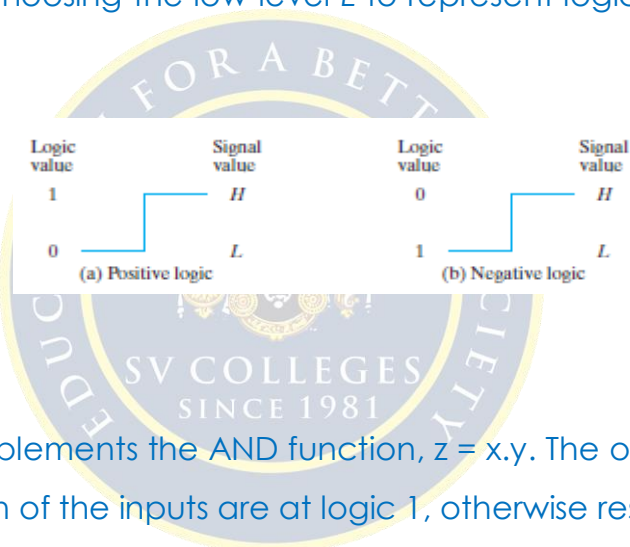
Name	Graphic symbol	Algebraic function	Truth table															
AND		$F = x \cdot y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = x + y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	
NAND		$F = (xy)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = (x + y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Positive and Negative Logic

The binary signal at the inputs and outputs of any gate has one of two values, except during transition. One signal value represents logic 1 and the other logic 0. Since two signal values are assigned to two logic values, there exist two different assignments of signal values to logic levels.

Positive Logic: Choosing the high-level H to represent logic 1 defines a positive logic system.

Negative Logic: Choosing the low-level L to represent logic 1 defines a negative logic system.



The AND Gate

The AND gate implements the AND function, $z = x.y$. The output of the gate is at logic 1 when both of the inputs are at logic 1, otherwise results in logic 0.

Logic Symbol



Truth Table

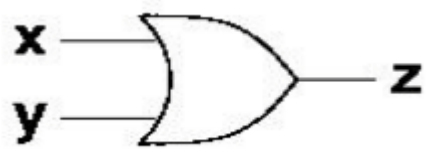
Inputs		Output
x	y	$z = x.y$
0	0	0
0	1	0
1	0	0
1	1	1

The AND function can have any number of inputs. However, for practical reasons, commercial AND gates are most commonly manufactured with 2, 3, or 4 inputs.

The OR Gate:

The OR gate implements the OR function, $z = x+y$. The output of the gate is at logic 1 when either of the inputs is at logic 1, otherwise results in logic 0.

Logic Symbol



Truth Table

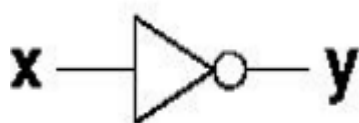
Inputs		Output
x	y	$z = x+y$
0	0	0
0	1	1
1	0	1
1	1	1

The OR function can have any number of inputs. However, for practical reasons, commercial OR gates are most commonly manufactured with 2, 3, or 4 inputs.

The NOT Gate or Inverter:

The NOT gate has only one input and one output. The output is complement of the input i.e., $y=x'$.

Logic Symbol



Truth Table

Input	Output
x	$y = x'$
0	1

1	0
---	---

The bubble at the output denotes the logical inversion.

Extension of gates to multiple inputs:

The AND, OR gates can be extended to multiple inputs as they satisfy both commutative and associative property. The NAND and NOR functions are commutative, but they are not associative (i.e., $(x \downarrow y) \downarrow z \neq x \downarrow (y \downarrow z)$), hence the definition of the operation is to be modified slightly to extend the gate to have more than two inputs. To overcome this difficulty, we define the multiple NOR (or NAND) gate as a complemented OR (or AND) gate. Thus, by definition, we have

$$x \downarrow y \downarrow z = (x + y + z)'$$

$$x \uparrow y \uparrow z = (xyz)'$$

3.4. Boolean Algebra

Boolean algebra was an algebraic system developed by George Boole in 1854. The two valued Boolean algebra is also known as Switching Algebra. Boolean algebra is an algebraic structure defined on a set of elements B, together with two binary operators (+, .) provided the following Huntington postulates are satisfied.

- (a) The structure is closed with respect to the operator +
 - (b) The structure is closed with respect to the operator .
- (a) The element 0 is an identity element with respect to +;
that is, $x + 0 = 0 + x = x$.
 - (b) The element 1 is an identity element with respect to . ;
that is, $x . 1 = 1 . x = x$.
- (a) The structure is commutative with respect to +; that is, $x + y = y + x$.

(b) The structure is commutative with respect to $.$; that is, $x.y = y.x$.

4. (a) The operator $.$ is distributive over $+$; that is, $x.(y+z) = (x.y) + (x.z)$.

(b) The operator $+$ is distributive over $.$; that is, $x+(y.z) = (x+y).(x+z)$.

5. For every element $x \in B$, there exists an element $x' \in B$ (called the complement of x) such that (a) $x+x' = 1$ and (b) $x.x' = 0$.

6. There exist at least two elements $x, y \in B$ such that $x \neq y$.

The two valued Boolean algebra:

The two valued Boolean algebra is defined on a set $B = \{0,1\}$ with two binary operators $+$ and $.$, whose rules are defined as below:

x	y	$x.y$
0	0	0
0	1	0
1	0	0
1	1	1

x	y	$x+y$
0	0	0
0	1	0
1	0	0
1	1	1

x	x'
0	1
1	0

1. That the structure is closed with respect to the two operators is obvious from the tables, since the result of each operation is either 1 or 0 and $1, 0 \in B$.

2. From the tables, we see that

(a) $0+0=0; 0+1=1+0=1$; (b) $1.1=1; 1.0=0.1=0$.

This establishes the two identity elements, 0 for $+$ and 1 for $.$ as defined by postulate 2.

3. The commutative laws are obvious from the symmetry of the binary operator tables.

4. (a) The distributive law $x.(y+z) = (x.y) + (x.z)$ can be shown to hold from the operator tables by forming a truth table of all possible values of x, y , and z . For

each combination, we derive $x \cdot (y + z)$ and show that the value is the same as the value of $(x \cdot y) + (x \cdot z)$:

(b) The *distributive* law of $+$ over $\cdot$ can be shown to hold by means of a truth table similar to the one in part (a).

5. From the complement table, it is easily shown that

(a) $x + x' = 1$, since $0 + 0' = 0 + 1 = 1$ and $1 + 1' = 1 + 0 = 1$.

(b) $x \cdot x' = 0$, since $0 \cdot 0' = 0 \cdot 1 = 0$ and $1 \cdot 1' = 1 \cdot 0 = 0$.

Thus, postulate 5 is verified.

x	y	z	y+z	x.(y+z)	x.y	x.z	x.y+x.z
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

1. Postulate 6 is satisfied because the two-valued Boolean algebra has two elements, 1 and 0, with $1 \neq 0$.

Basic Theorems and Properties and Boolean algebra:

Duality: The important property of Boolean algebra is duality principle and states that every algebraic expression deducible from the postulates of Boolean algebra remains valid if the operators and identity elements are interchanged. The dual of an algebraic expression can be obtained by simply interchanging OR and AND operators and replacing 1's by 0's and 0's by 1's.

Postulate 2	(a) $x + 0 = x$	(b) $x \cdot 1 = x$
Postulate 3, Commutative	(a) $x + y = y + x$	(b) $x \cdot y = y \cdot x$
Postulate 4, Distributive	(a) $x \cdot (y + z) = x \cdot y + x \cdot z$	(b) $x + yz = (x + y) \cdot (x + z)$
Postulate 5	(a) $x + x' = 1$	(b) $x \cdot x' = 0$
Theorem 1	(a) $x + x = x$	(b) $x \cdot x = x$
Theorem 2	(a) $x + 1 = 1$	(b) $x \cdot 0 = 0$

Theorem 3, Involution

$$(x')' = x$$

Theorem 4, Associative

$$(a) \quad x + (y + z) = (x + y) + z \quad (b) \quad x(yz) = (xy)z$$

Theorem 5, DeMorgan

$$(a) \quad (x + y)' = x'y' \quad (b) \quad (xy)' = x' + y'$$

Theorem 6, Absorption

$$(a) \quad x + xy = x \quad (b) \quad x(x + y) = x$$

Theorem Proofs:**THEOREM 1(a):** $x + x = x$.**Statement**

$$\begin{aligned} x + x &= (x + x) \cdot 1 \\ &= (x + x)(x + x') \\ &= x + xx' \\ &= x + 0 \\ &= x \end{aligned}$$

Justification

$$\begin{aligned} x \cdot 1 &= x, \text{ postulate 2(b)} \\ x + x' &= 1, \text{ postulate 5(a)} \\ (x + y) \cdot (x + z) &= x + yz, \text{ postulate 4(b)} \\ x \cdot x' &= 0, \text{ postulate 5(b)} \\ x + 0 &= x, \text{ postulate 2(a)} \end{aligned}$$

THEOREM 1(b): $x \cdot x = x$.**Statement**

$$\begin{aligned} x \cdot x &= xx + 0 \\ &= xx + xx' \\ &= x(x + x') \\ &= x \cdot 1 \\ &= x \end{aligned}$$

Justification

$$\begin{aligned} x + 0 &= x, \text{ postulate 2(a)} \\ x \cdot x' &= 0, \text{ postulate 5(b)} \\ x \cdot y + x \cdot z &= x \cdot (y + z), \text{ postulate 4(a)} \\ x + x' &= 1, \text{ postulate 5(a)} \\ x \cdot 1 &= x, \text{ postulate 2(b)} \end{aligned}$$

Note that theorem 1(b) is the dual of theorem 1(a) and that each step of the proof in part (b) is the dual of its counterpart in part (a). Any dual theorem can be similarly derived from the proof of its corresponding theorem.

THEOREM 2(a): $x + 1 = 1$.**Statement**

$$\begin{aligned} x + 1 &= (x + 1) \cdot 1 \\ &= (x + 1)(x + x') \\ &= x + 1 \cdot x' \\ &= x + x' \\ &= 1 \end{aligned}$$

Justification

$$\begin{aligned} x \cdot 1 &= x, \text{ postulate 2(b)} \\ x + x' &= 1, \text{ postulate 5(a)} \\ (x + y) \cdot (x + z) &= x + yz, \text{ postulate 4(b)} \\ x \cdot 1 &= x, \text{ postulate 2(b)} \\ x + x' &= 1, \text{ postulate 5(a)} \end{aligned}$$

THEOREM 2(b): $x \cdot 0 = 0$. (Can be proved by duality)**Operator Precedence**

The operator precedence for evaluating Boolean expressions is

- (1) Parentheses,
- (2) NOT,
- (3) AND, and
- (4) OR.

In other words, expressions inside parentheses must be evaluated before all other operations. The next operation that holds precedence is the complement, and then follows the AND and, finally, the OR.

THEOREM 4(a):

The validity of the associative law and DeMorgan's theorem can be easily verified with truth tables. The truth table for the Associative law, $x+(y+z)=(x+y)+z$, is as follows:

x	y	z	$(y+z)$	$x+(y+z)$	$(x+y)$	$(x+y)+z$
0	0	0	0	0	0	0
0	0	1	1	1	0	1
0	1	0	1	1	1	1
0	1	1	1	1	1	1
1	0	0	0	1	1	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

THEOREM 5(a):

The validity of the associative law and DeMorgan's theorem can be easily verified with truth tables. The truth table for the DeMorgan's theorem, $(x + y)' = x'y'$, is as follows:

x	y	$x+y$	$(x+y)'$	x'	y'	$x' \cdot y'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

THEOREM 6(a): $x + xy = x$.

Statement

$$\begin{aligned}
 x + xy &= x \cdot 1 + xy \\
 &= x(1 + y) \\
 &= x(y + 1) \\
 &= x \cdot 1 \\
 &= x
 \end{aligned}$$

Justification

$$\begin{aligned}
 x \cdot 1 &= x \\
 xy + xz &= x(y+z) \\
 x+y &= y+x \\
 x+1 &= 1 \\
 x \cdot 1 &= x
 \end{aligned}$$

THEOREM 6(b): $x(x + y) = x$ by duality.

3.5. BOOLEAN FUNCTIONS

A Boolean function described by an algebraic expression consists of binary variables, the constants 0 and 1, and the logic operation symbols. For a given value of the binary variables, the function can be equal to either 1 or 0.

$$\text{Ex: } F = x + y'z$$

A Boolean function can be represented in a truth table. The number of rows in the truth table is 2^n , where n is the number of variables in the function. The binary combinations for the truth table are obtained from the binary numbers by counting from 0 through $2^n - 1$. Following Table shows the truth table for the function F .

x	y	z	y'	$y'z$	$F = x + y'z$
0	0	0	1	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	1	0	1
1	0	1	1	1	1
1	1	0	0	0	1
1	1	1	0	0	1

When a Boolean expression is implemented with logic gates, each term requires a gate and each variable within the term designates an input to the gate. We define a *literal* to be a single variable within a term, in complemented or un-complemented form. By manipulating a Boolean expression according to the rules of Boolean algebra, it is sometimes possible to obtain a simpler expression for the same function and thus reduce the number of gates in the circuit and the number of inputs to the gate which in turn reduce the circuit complexity and also the cost of the circuit.

Consider, for example, the following Boolean function:

$$F = x'y'z + x'yz + xy'.$$

Now consider the possible simplification of the function by applying some of the identities of Boolean algebra:

$$F = x'y'z + x'yz + xy' = x'z(y' + y) + xy' = x'z + xy'.$$

The function is reduced to only two terms.

3.6. Two-Level NAND and NOR Implementation Of Boolean Functions

In addition to AND, OR, and NOT gates, other logic gates like NAND and NOR are also used in the design of digital circuits.

The NAND gate represents the complement of the AND operation. Its name is an abbreviation of NOT AND. The graphic symbol for the NAND gate consists of an AND symbol with a bubble on the output, denoting that a complement operation is performed on the output of the AND gate as shown earlier.

The NOR gate represents the complement of the OR operation. Its name is an abbreviation of NOT OR. The graphic symbol for the NOR gate consists of an OR symbol with a bubble on the output, denoting that a complement operation is performed on the output of the OR gate as shown earlier.

The NAND and NOR gates are universal gates. A universal gate is a gate which can implement any Boolean function without any need of other gates. NAND and NOR gates are economical and easier to fabricate and are the basic gates used in all IC digital logic families.

Realization of basic gates using NAND Gate :

To prove that any Boolean function can be implemented using only NAND gates, we need to show that the AND, OR, and NOT operations can be performed using only these gates.

Implementing an Inverter Using only NAND Gate:

The figure shows two ways in which a NAND gate can be used as an inverter (NOT gate).

1. All NAND input pins connect to the input signal A gives an output A'

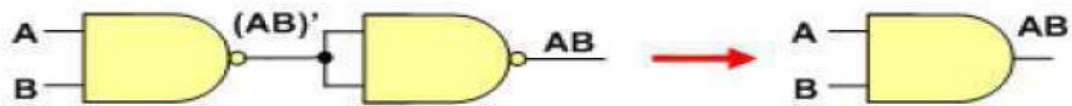


2. One NAND input pin is connected to the input signal A while all other input pins are connected to logic 1. The output will be A'.



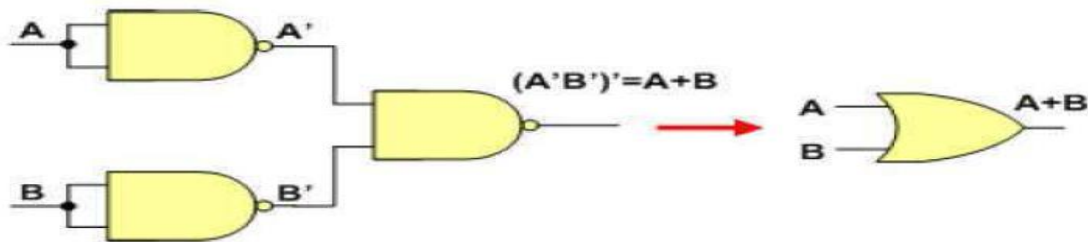
Implementing AND Using only NAND Gates:

The AND is replaced by a NAND gate with its output complemented by a NAND gate inverter, which is as shown in the figure.



Implementing OR Using only NAND Gates:

The OR gate is replaced by a NAND gate with all its inputs complemented by NAND gate inverters, which is shown in figure below:



Thus, the NAND gate is a universal gate since it can implement the AND, OR and NOT functions.

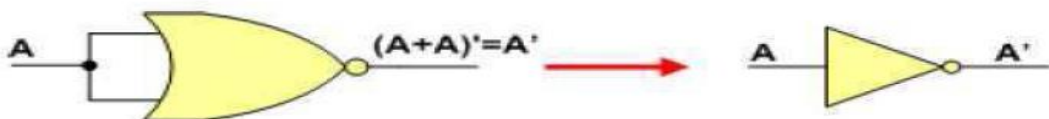
Realization of basic gates using NOR Gate:

To prove that any Boolean function can be implemented using only NOR gates, we need to show that the AND, OR, and NOT operations can be performed using only NOR gates.

Implementing Inverter Using only NOR Gate:

The figure shows two ways in which a NOR gate can be used as an inverter (NOT gate).

1. All NOR input pins connect to the input signal A gives an output A'.

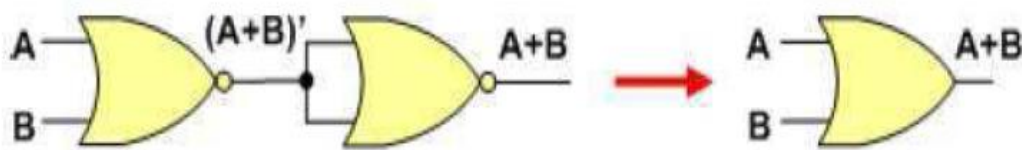


2. One NOR input pin is connected to the input signal A while all other input pins are connected to logic 0. The output will be A'.



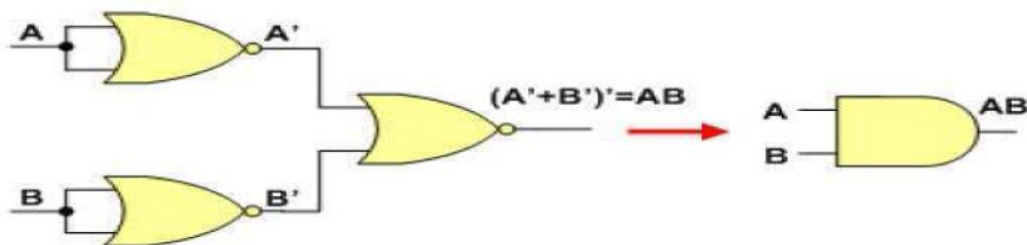
Implementing OR Using only NOR Gates:

The OR is replaced by a NOR gate with its output complemented by a NOR gate inverter as shown in figure below:



Implementing AND Using only NOR Gates:

The AND gate is replaced by a NOR gate with all its inputs complemented by NOR gate inverters, which is as shown below:



Thus, the NOR gate is a universal gate since it can implement the AND, OR and NOT functions.

Equivalent Gates:

The shown figure summarizes important cases of gate equivalence. Note that bubbles indicate a complement operation (inverter).

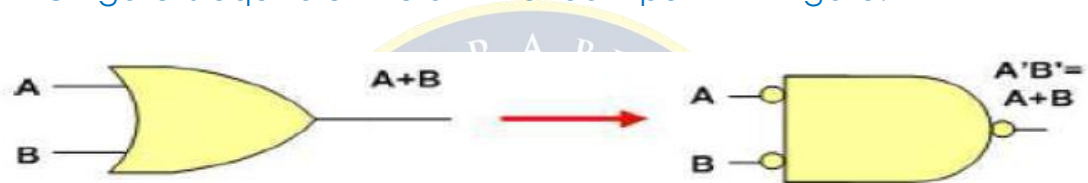
A NAND gate is equivalent to an inverted-input OR gate (INVERT – OR).



An AND gate is equivalent to an inverted-input NOR gate.



An OR gate is equivalent to an inverted-input NAND gate.



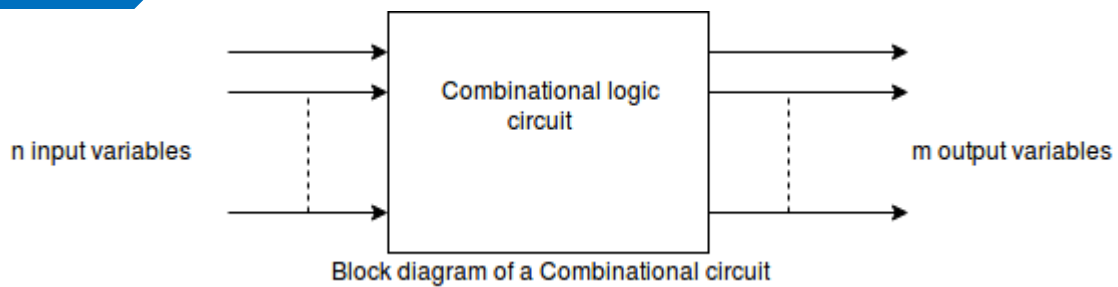
Two NOT gates in series are same as a buffer because they cancel each other as $A''=A$.



3.7. Combinational Circuits

- A combinational circuit comprises of logic gates whose outputs at any time are determined directly from the present combination of inputs without any regard to previous inputs.
- A combinational circuit performs a specific information-processing operation fully specified logically by a set of Boolean functions.

The basic components of a combinational circuit are: input variables, logic gates, and output variables.



- The 'n' input variables come from an external source whereas the 'm' output variables go to an external destination. In many applications, the source or destination are storage registers.

3.7.1 Design procedure of a Combinational Circuit

The design procedure of a combinational circuit involves the following steps:

1. The problem is stated.
2. The total number of available input variables and required output variables is determined.
3. The input and output variables are allocated with letter symbols.
4. The exact truth table that defines the required relationships between inputs and outputs is derived.
5. The simplified Boolean function is obtained from each output.
6. The logic diagram is drawn.

The combinational circuit that performs the addition of two bits is called a half adder and the one that performs the addition of three bits (two significant bits and a previous carry) is a full adder.

3.7.2 Half - Adder

A Half-adder circuit needs two binary inputs and two binary outputs. The input variable shows the augend and addend bits whereas the output variable produces the sum and carry. We can understand the function of a half-adder by formulating a truth table. The truth table for a half-adder is:

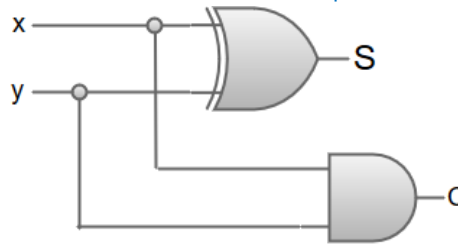
x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

- 'x' and 'y' are the two inputs, and S (Sum) and C (Carry) are the two outputs.
- The Carry output is '0' unless both the inputs are 1.
- 'S' represents the least significant bit of the sum.

The simplified sum of products (SOP) expressions is:

- $S = x'y + xy'$, $C = xy$

The logic diagram for a half-adder circuit can be represented as:



$$S = x'y + xy'$$

$$C = xy$$

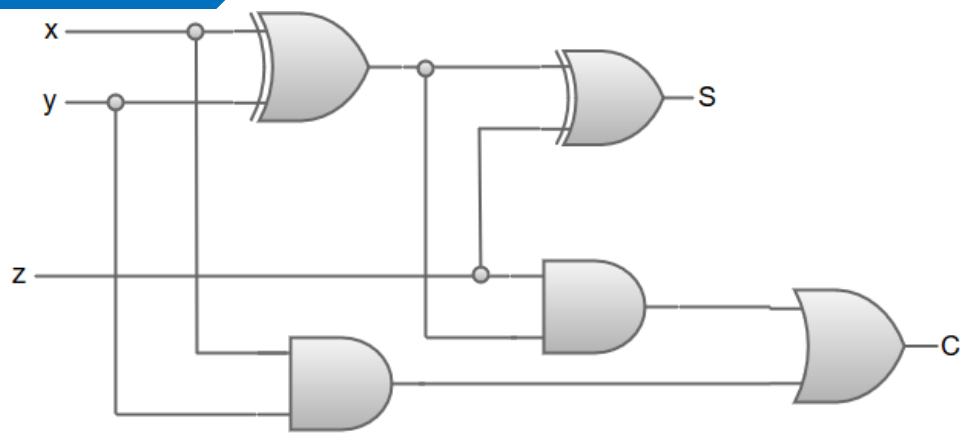
3.7.3 Full - Adder

This circuit needs three binary inputs and two binary outputs. The truth table for a full-adder is:

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- Two of the input variable 'x' and 'y', represent the two significant bits to be added.
- The third input variable 'z', represents the carry from the previous lower significant position.
- The outputs are designated by the symbol 'S' for sum and 'C' for carry.
- The eight rows under the input variables designate all possible combinations of 0's, and 1's that these variables may have.
- The input-output logical relationship of the full-adder circuit may be expressed in two Boolean functions, one for each output variable.
Each output Boolean function can be simplified by using a unique map method.

The logic diagram for a full-adder circuit can be represented as:



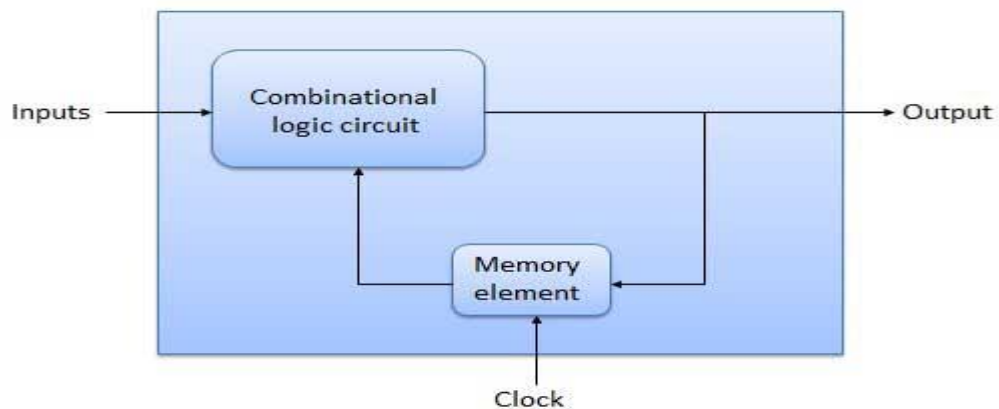
$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$C = xy + xz + yz$$

3.8. Sequential Circuits

The combinational circuit does not use any memory. Hence the previous state of input does not have any effect on the present state of the circuit. But sequential circuit has memory so output can vary based on input. This type of circuits uses previous input, output, clock and a memory element.

Block diagram



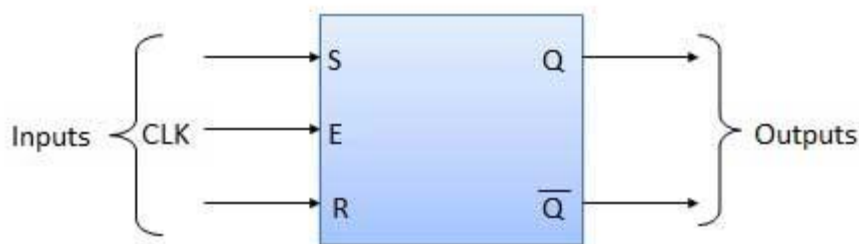
3.8.1 Flip Flop

Flip flop is a sequential circuit which generally samples its inputs and changes its outputs only at particular instants of time and not continuously. Flip flop is said to be edge sensitive or edge triggered rather than being level triggered like latches.

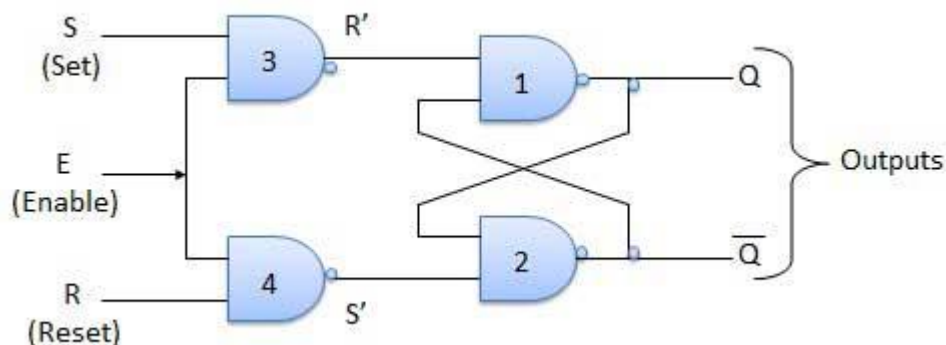
S-R Flip Flop

It is basically S-R latch using NAND gates with an additional **enable** input. It is also called as level triggered SR-FF. For this, circuit in output will take place if and only if the enable input (E) is made active. In short this circuit will operate as an S-R latch if $E = 1$ but there is no change in the output if $E = 0$.

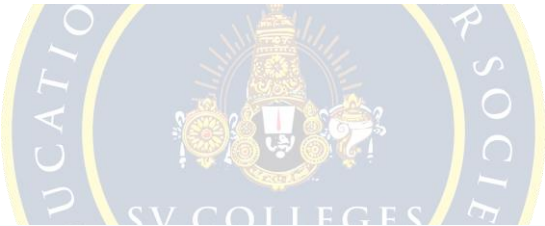
Block Diagram



Circuit Diagram



Truth Table

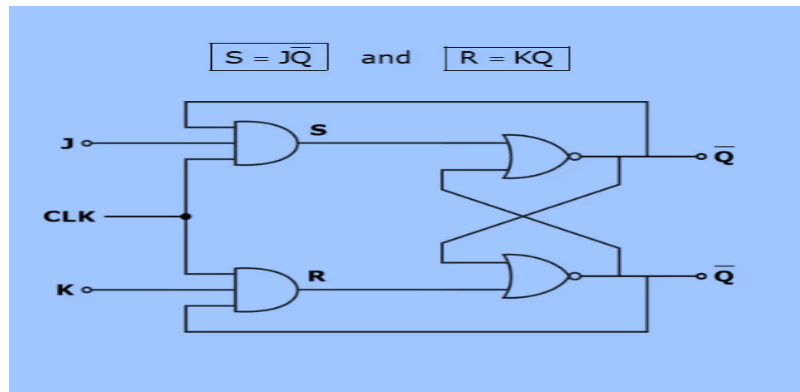


Inputs			Outputs		Comments
E	S	R	Q_{n+1}	$\overline{Q}_{n+1}$	
1	0	0	Q_n	$\overline{Q}_n$	No change
1	0	1	0	1	Rset
1	1	0	1	0	Set
1	1	1	x	x	Indeterminate

JK Flip Flop

JK Flip Flop is one of the most used flip-flops in digital circuits. The universal flip flop has two inputs, 'J' and 'K.' The JK Flip Flop is a gated SR Flip-Flop with a clock input circuitry that prevents the illegal or invalid output when both inputs S and R are equal to logic level "1."

Circuit Diagram



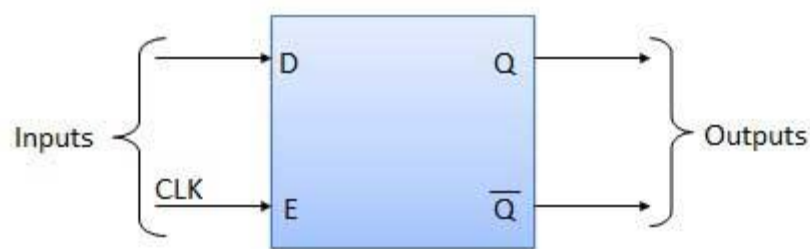
Truth Table

Inputs			Outputs		Comments
E	J	K	Q_{n+1}	$\bar{Q}_{n+1}$	
1	0	0	Q_n	$\bar{Q}_n$	No change
1	0	1	0	1	Rset
1	1	0	1	0	Set
1	1	1	$\bar{Q}_n$	Q_n	Toggle

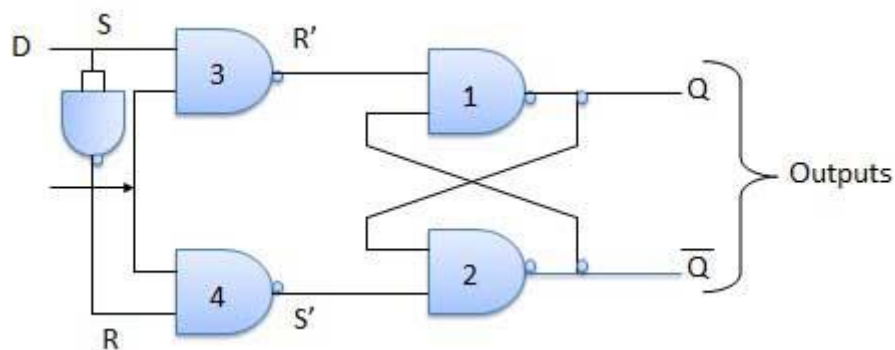
Delay Flip Flop / D Flip Flop

Delay Flip Flop or D Flip Flop is the simple gated S-R latch with a NAND inverter connected between S and R inputs. It has only one input. The input data is appearing at the output after some time. Due to this data delay between i/p and o/p, it is called delay flip flop. S and R will be the complements of each other due to NAND inverter. Hence $S = R = 0$ or $S = R = 1$, these input condition will never appear. This problem is avoid by $SR = 00$ and $SR = 11$ conditions.

Block Diagram



Circuit Diagram



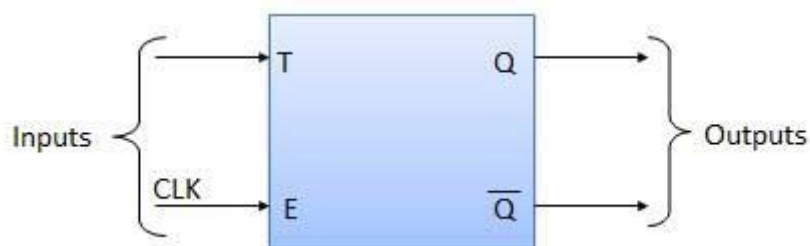
Truth Table

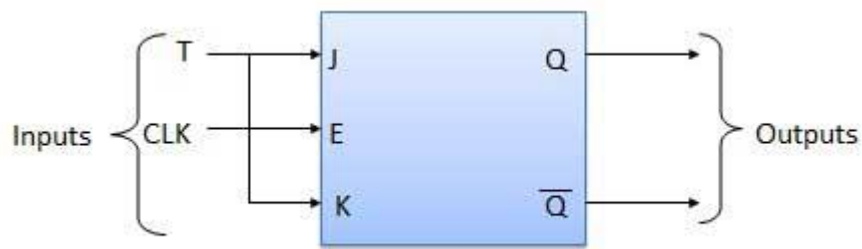
Inputs		Outputs		Comments
E	D	Q_{n+1}	$\overline{Q}_{n+1}$	
1	0	0	1	Rset
1	1	1	0	Set

Toggle Flip Flop / T Flip Flop

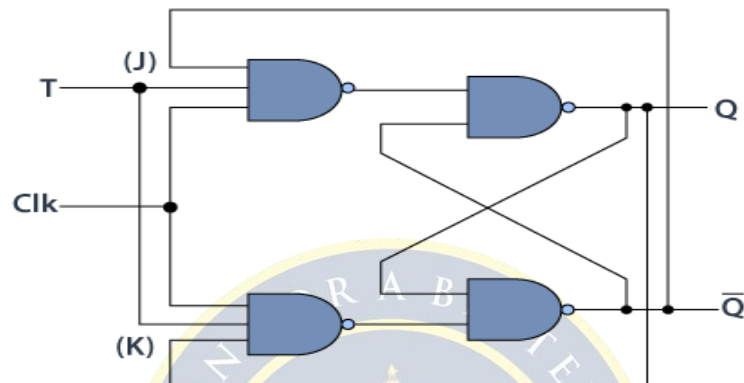
Toggle flip flop is basically a JK flip flop with J and K terminals permanently connected together. It has only input denoted by **T** as shown in the Symbol Diagram. The symbol for positive edge triggered T flip flop is shown in the Block Diagram.

Block Diagram





Circuit Diagram

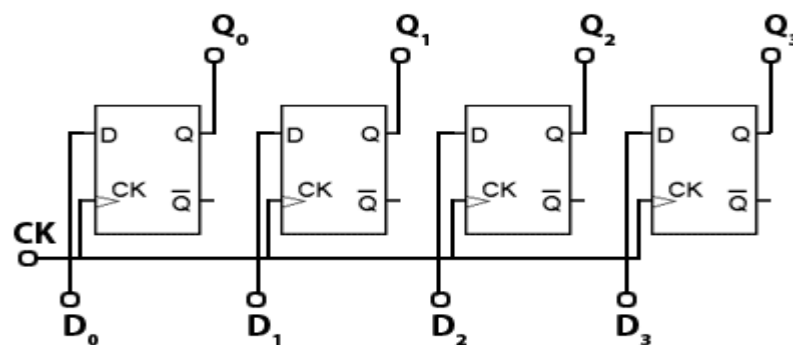


Truth Table

Inputs		Outputs		Comments
E	T	Q_{n+1}	$\overline{Q}_{n+1}$	
1	0	Q_n	$\overline{Q}_n$	No change
1	1	$\overline{Q}_n$	Q_n	Toggle

3.8.2 Digital Registers

Flip-flop is a 1 bit memory cell which can be used for storing the digital data. To increase the storage capacity in terms of number of bits, we have to use a group of flip-flop. Such a group of flip-flop is known as a **Register**. The **n-bit register** will consist of **n** number of flip-flop and it is capable of storing an **n-bit** word.



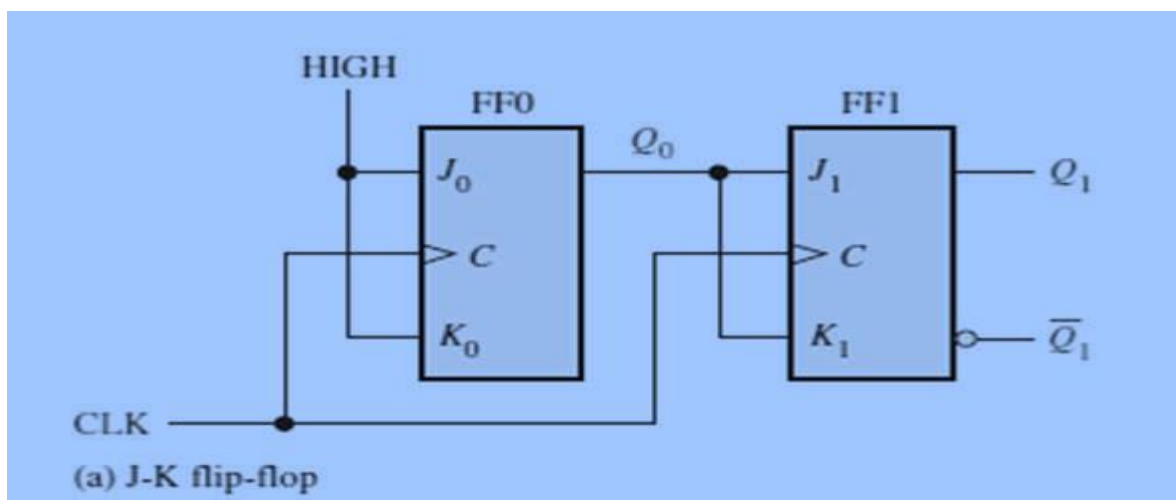
3.8.3 Digital Counter

Counter is a sequential circuit. A digital circuit which is used for counting pulses is known counter. Counter is the widest application of flip-flops. It is a group of flip-flops with a clock signal applied. Counters are of two types.

- Synchronous.
- Asynchronous or ripple counters.

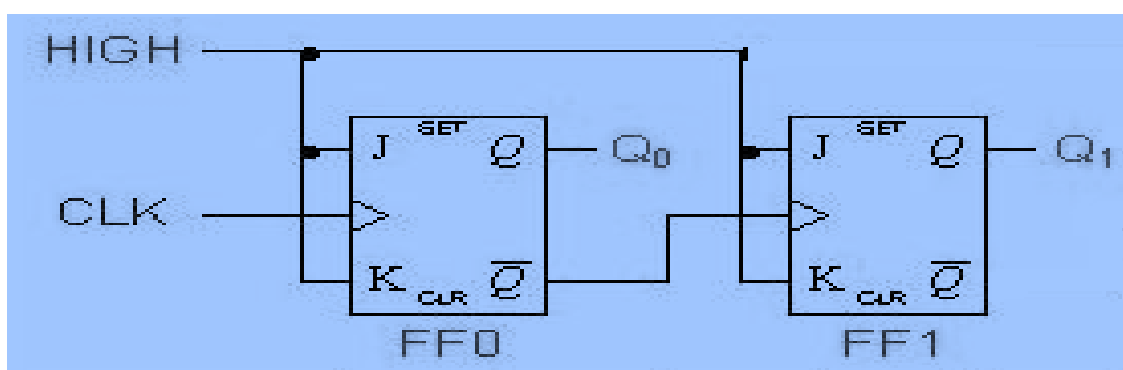
Synchronous Counter:

If the "clock" pulses are applied to all the flip-flops in a counter simultaneously, then such a counter is called as synchronous counter.



Asynchronous or ripple counter:

The JK flip-flop is used with J and K connected permanently to logic 1. External clock is applied to the clock input of flip-flop 0 (LSB) and Q_0 output is applied to the clock input of the next flip-flop i.e. FF-1 (MSB).



9. Practice Quiz

1. The binary number system has the Radix of

- (a) 8
- (b) 2**
- (c) 4
- (d) 16

2. Group of 8 bits is called

- (a) byte**
- (b) bit
- (c) nibble
- (d) data

3. In the base r system, the distinct symbols ranges from

- (a) 1 to r
- (b) 0 to $(r-1)$**
- (c) 0 to $2r$
- (d) 0 to $r/2$

4. Which of the following is incorrect regarding Gray Code?

- (a) Weighted Code**
- (b) Unit Distance Code
- (c)Cyclic Code
- (d) Has reflection property

7. The logic gate whose output becomes zero, if any one of the inputs is zero is called

- (a) OR gate
- (b) NOR gate
- (c)AND gate**
- (d) NAND gate

8. The logic gate whose output will be one, if both inputs are same is called

- (a) OR gate
- (b) XOR gate
- (c)AND gate**

(d) XNOR gate

8. The logic gate whose output will be one, if any one of the inputs is one is called

(a) AND gate

(b) OR gate

(c) NAND gate

(d) NOR gate

9. Which of the following operation is not available in Boolean Algebra?

(a) Division

(b) Complement

(c) Addition

(d) Multiplication

10. The Absorption law states that

(a) $x + xy = x$

(b) $x + x = x$

(c) $x \cdot x = x$

(d) $x \cdot x' = 0$

11. Dual of $x + y$ is

(a) $x \cdot y$

(b) $x' + y'$

(c) $x' \cdot y'$

(d) $x \cdot y'$

12. The De Morgan's theorem states that

(a) $(x + y)' = x' y'$

(b) $x + (y + z) = (x + y) + z$

(c) $x + yz = (x + y)(x + z)$

(d) $(x')'$

13. The binary equivalent of hexadecimal number 7BD is

(a) 011110111101

(b) 111010111010

(c) 111101111010

(d) 101110101110



14. Which of the following 4-bit combination is invalid in the BCD code?

- (a) 0010
- (b) 1010**
- (c) 0101
- (d) 1001

15. Which of the following Number System contains alphabets?

- (a) Binary Number System
- (b) Octal Number System
- (c) Decimal Number System
- (d) Hexa Decimal Number System**

16. Which of the following gates are added to the inputs of The OR gate to convert it into NAND gate?

- a) OR
- b) NOT**
- c) AND
- d) X-OR

17. $AB + A'B + 1 =$

- a) B
- b) 0
- c) 1**
- d) A

18.. What is the minimum number of 2-input NOR gates required to realize an X-OR gate

- a) 5**
- b) 3
- c) 4
- d) 6

19. The number of inputs and outputs of a full adder are

- a) 3 and 2 respectively**
- b) 2 and 3 respectively
- c) 4 and 2 respectively
- d) 2 and 4 respectively

20. A flip flop is a

- a) combinational circuit**
- b) **memory element**
- c) arithmetic element
- d) memory or arithmetic element

21. Which of the following is used as switch?

- a) J-K flip-flop**
- b) S-R flip-flop



- c) T-flip-flop
d) **D-flip-flop**

22. T- FF used as a

- a) transfer Data switch
b) **toggle switch**
c) time delay switch
d) none

23. In a synchronous counter all flip flops are clocked together.

- a) **True**
b) False

10. Assignments

S.No	Question	BL	CO
1	Explain the functionality of various logic gates using their truth tables.	2	1
2	Convert the following numbers with the indicated bases to decimal: (a)(3410) ₅ (b)(891) ₁₂ (c)(246) ₈	3	1
3	Write short notes on Theorems and Postulates used in Boolean Algebra.	2	1
4	Write short notes on the following binary codes (a)BCD (b) Excess 3 (c) Gray Code	2	1
5	Explain the functionality of Half Adder & Full Adder using their truth tables.	2	1
6	Explain the functionality of SR ,JK, T and D flip flops using their truth tables.	2	1
7	Draw the Schematic diagrams of Digital Register using D Flip flop & Digital counter using JK flip flop.	2	1

11. Part A- Question & Answers

S.No	Question& Answers	BL	CO
1	Convert (1101)₂ into a decimal number. Ans. $1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$ $= 8 + 4 + 0 + 1$ $= 13$	3	1
2	Convert (89)₁₆ into a binary number. Ans. $8 = 1000$ and $9 = 1001$ Therefore, $(89)_{16} = (10001001)_2$	3	1
3	What is a Logic gate? Ans. Logic gates are the basic elements that make up a digital system. The electronic gate is a circuit that is able to operate on a number of binary inputs in order to perform a particular logical function.	2	1
4	Define duality property. Ans. Duality property states that every algebraic expression deducible from the postulates of Boolean algebra remains valid if the operators and identity elements are interchanged. If the dual of an algebraic expression is desired, we simply interchange OR and AND operators and replace 1's by 0's and 0's by 1's.	2	1
5	State De Morgan's theorem. Ans. DeMorgan suggested two theorems that form important part of Boolean algebra. They are, 1. The complement of a product is equal to the sum of the complements. $(AB)' = A' + B'$ 2. The complement of a sum term is equal to the product of the complements. $(A+B)' = A'B'$		
6	State the associative property of boolean algebra. Ans. The associative property of Boolean algebra states that the Oring of several variables results in the same regardless of the grouping of the variables. The associative property is stated as follows: $A + (B+C) = (A+B) + C$	2	1

7	What are basic properties of Boolean algebra? Ans. The basic properties of Boolean algebra are commutative property, associative property and distributive property.	2	1																																				
8	Write the truth tables of NAND and NOR gates. Ans. NAND Gate <table><tr><th colspan="2">Inputs</th><th>Output</th></tr><tr><th>x</th><th>y</th><th>(x.y)'</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table> NOR Gate <table><tr><th colspan="2">Inputs</th><th>Output</th></tr><tr><th>x</th><th>y</th><th>(x+y)'</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Inputs		Output	x	y	(x.y)'	0	0	1	0	1	1	1	0	1	1	1	0	Inputs		Output	x	y	(x+y)'	0	0	1	0	1	0	1	0	0	1	1	0	2	1
Inputs		Output																																					
x	y	(x.y)'																																					
0	0	1																																					
0	1	1																																					
1	0	1																																					
1	1	0																																					
Inputs		Output																																					
x	y	(x+y)'																																					
0	0	1																																					
0	1	0																																					
1	0	0																																					
1	1	0																																					
9	What are combinational circuits? A combinational circuit consists of logic gates whose outputs at any time are determined from the present combination of inputs. A combinational circuit performs an operation that can be specified logically by a set of Boolean functions. It consists of input variables, logic gates, and output variables.	1	1																																				
10	Define half adder. A combinational circuit that performs the addition of two bits is called a half adder. A half adder needs two binary inputs and two binary outputs. The input variables designate the augend and addend bits; the output variables produce the sum and carry	4	3																																				

12. Part B- Questions

S.No	Question	BL	CO
1	(a) Explain any two binary codes with example. (b) Each of the following arithmetic operations is correct in at least one number system. Determine the possible bases of the numbers in each operation: (i) $1234 + 5432 = 6666$ (ii) $41/3 = 13$ (iii) $33/3 = 11$	2, 3	1

2	a. Simplify the following algebraic expression: $(x' + xyz') + (x' + xyz')(x + x'y'z)$ b. Prove that if $w'x + yz' = 0$, then $wx + y'(w' + z') = wx + xz + x'z' + w'y'z$ c. Determine the canonical sum-of-products representation of the following function: $f(x, y, z) = z + (x' + y)(x + y')$	4	1
3	a. Perform the subtraction with the following decimal numbers using 10's complement and 9's complement: (i) $5250 - 321$. (ii) $20 - 1000$. b. Prove the Boolean identity $yx + yz = (x + y)(x + z)$.	3	1
4	a. Convert the following: (i) $(163.789)_{10} = ()_8$. (ii) $(101101110001.00101)_2 = ()_8$. (iii) $(292)_{16} = ()_2$. b. Prove that AND-OR network is equivalent to NAND-NAND network.	3	1
5	a. Represent decimal number 6027 in: (i) BCD. (ii) excess-3 code. (iii) 2421 code. b. Expand $A(A+B)$ to maxterms and minterms.	3	1
6	What is a combinational circuit? Explain with one example.	1	1
7	Design a full adder with two half adders and OR gate.	4	3
8	Design a Asynchronous counter using T flip-flop	4	3

13. Supportive Online Certification Courses

1. Course on "Digital Electronic Circuits" by prof. Goutham saha conducted by IIT, Kharagpur – 12 weeks.

14. Real Time Applications

S.No	Applications	CO
1	Digital Systems like computers, Digital devices, etc.	2
2	Switches	2
3	Memories	2

4	Seven Segment Display using LED	2
5	Digital Communication systems	2

15. Contents Beyond the Syllabus

Error-Detecting Codes:

During the process of binary data transmission errors may occur. Error detecting codes are used to detect such type of errors. If a single error transforms a valid code word into invalid one, it is said to be a single error detecting code.

A parity bit is an extra bit included with the n-bit binary message to make the total number of 1's in the message (including the parity bit) either odd or even.

In the even parity method, parity bit (0 or 1) is added such that the total number of 1's in the coded group must be even (including parity bit). Similarly, odd in odd parity method.

Three bit message x y z	Even Parity Method P x y z	Odd Parity Method P x y z
000	0 000	1 000
001	1 001	0 010
010	1 010	0 010
011	0 011	1 011
100	1 100	0 100
101	0 101	1 101
110	0 110	1 110
111	1 111	0 111

Parity check is commonly used for error detection. Even though parity code is meant for single error detection, it can detect any odd number of errors, but it is not possible to correct. Even number of errors cannot be detectable using parity method.

2 out of 5 code:

The 2 out of 5 code consists of all 10 possible combinations of two 1's in a 5-bit code word. With the exception of the code word for decimal 0, the 2 out of 5 code

is a weighted code with weights 0 1 2 4 7. If a single error occurs, it transforms a valid code word into an invalid one, and making error detection straight forward.

Decimal digit	2 out of 5 code
	0 1 2 4 7
0	0 0 0 1 1
1	1 1 0 0 0
2	1 0 1 0 0
3	0 1 1 0 0
4	1 0 0 1 0
5	0 1 0 1 0
6	0 0 1 1 0
7	1 0 0 0 1
8	0 1 0 0 1
9	0 0 1 0 1

Error Correcting Codes:

A code is said to be an error correcting code, if the correct code word can always be deduced from the erroneous word. The minimum distance must be 3 for error correcting codes, which can correct single error and detect double errors. The distance between two code words is defined as the number of bits that must change in one code word so that another code word results. For example, the distance between 1001 and 1010 is 2, since two bits are to be changed. The minimum distance of a code is the smallest number of bits in which any two code words differ.

Hamming Code:

R.W. Hamming developed a system that provides a methodical way to add parity bits to the data bits in order to detect and correct errors, which is familiar as Hamming Code.

The basic principles in constructing a Hamming error correcting code are as follows:

- To each group of ' m ' information or message bits. ' k ' parity bits are added to form an $(m+k)$ bit code.

- The location of each of the $(m+k)$ bits within a code word is assigned a decimal value, starting by assigning a 1 to the Most Significant Bit (MSB) and $(m+k)$ to the Least Significant Bit (LSB).
- The value of k is selected such that it must satisfy the inequality $2^k \geq m+k+1$
- The parity bits are included in the positions 1, 2, 4, 2^{k-1}
- k parity bits are added for the selected bits of each code word so as to get either even or odd parity.

The process of constructing a Hamming code for message 0100 is explained as follows:

- The given message is 0100.
- Number of message bits $m=4$.
- Number of parity bits required is determined by trial and error method, which satisfies the inequality $2^k \geq m+k+1$
- $k = 3$ satisfies the inequality for the given 4-bit message. ($2^3 \geq 4+3+1$)
- The total number of bits in hamming code is
4 message bits + 3 parity bits = 7 bits.
- The bit positions are from 1 to 7, and 3 parity bits are included in positions 1, 2, 4

Bit Positions	1	2	3	4	5	6	7
Bit Designations	P_1	P_2	M_1	P_4	M_2	M_3	M_4
Original message			0		1	0	0
P_1 is to set even parity in the bit positions 1, 3, 5, 7	1		0		1		0
P_2 is to set even parity in the bit positions 2, 3, 6, 7		0	0			0	0
P_4 is to set even parity in the bit positions 4, 5, 6, 7				1	1	0	0
Coded Message	1	0	0	1	1	0	0

The generated hamming code for the given message is 1001100.

Error correction procedure using Hamming coded data:

- At the receiver side, k parity checks are performed on selected bits of each code word for either even or odd parity (depending on the parity method adopted at the transmitter).

- The result of each parity check is recorded as 1 or 0, depending on whether an error has or has not detected. These parity checks results $C_1, C_2 \dots C_k$ form a binary number, whose value is equal to the decimal value assigned to the location of the erroneous bit, and is equal to zero if no error occurs.

- The corrected code word can be obtained

by complementing the bit in the corresponding bit position.

- Since C_1 bit is '1' for errors in positions

1, 3, 5, 7 P_1 is selected so as to establish even/ odd parity in positions 1, 3, 5, 7.

- Since C_2 bit is '1' for errors in positions

2, 3, 6, 7 P_2 is selected so as to establish even/ odd parity in positions 2, 3, 6, 7.

- Since C_4 bit is '1' for errors in positions

4, 5, 6, 7 P_4 is selected so as to establish even/ odd parity in positions 4, 5, 6, 7.

Error Position	Position Number		
	C_4	C_2	C_1
0 (No error)	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

The process of correcting the error in the received Hamming code 1011100 with even parity:

Bit Positions	1	2	3	4	5	6	7	Parity check results
Bit Designations	P_1	P_2	M_1	P_4	M_2	M_3	M_4	

Message Received	1	0	0	1	1	1	0	
Even parity check C_1 in the bit positions 1, 3, 5, 7	1		0		1		0	0, no error
Even parity check C_2 in the bit positions 2, 3, 6, 7		0	0			1	0	1, error occurred
Even parity check C_4 in the bit positions 4, 5, 6, 7				1	1	1	0	1, error occurred
Corrected Message	1	0	0	1	1	0	0	

The parity check results $C_4 C_2 C_1$ represents the bit position with error i.e., 110. So the bit in the 6th position is complemented to get the correct message.

Hamming codes with even parity for BCD

Decimal digit	BCD	Hamming Code word
0	0000	0000000
1	0001	1101001
2	0010	0101010
3	0011	1000011
4	0100	1001100
5	0101	0100101
6	0110	1100110
7	0111	0001111
8	1000	1110000
9	1001	0011001

Alpha Numeric Codes

Many applications of digital computers require the handling not only of numbers, but also of other characters or symbols, such as the letters of the alphabet. An alphanumeric character set is a set of elements that includes the 10 decimal digits, the

26 letters of the alphabet, and a number of special characters. The commonly used alphanumeric codes are:

- ASCII
- EBCDIC
- Hollerith

ASCII:

- ASCII stands for American Standard Code for Information Interchange.
- It is a standard binary code with 7 bits used for representing alphanumeric character set of 128 symbols.
- It is generally represented using 8 bits, with one bit unused, which is reserved for future extension by setting MSB to 0, or can be used as parity bit.
- The ASCII code contains – 94 graphic characters, which can be printed and 34 non printable characters.
- The 94 graphic characters include –
 - 26 upper case letters (A to Z)
 - 26 lower case letters (a to z)
 - 10 decimal digits (0 to 9)
 - 32 special printable characters, such as %, &, etc.
- The 34 control characters are designated in the ASCII table with abbreviated names, which is used as control functions.
- Three types of control characters are
 - Format effectors – controls the layout of printing – Back space, enter, tab, etc.
 - Information separators – to separate the data into divisions – paragraphs, page breaks, etc.
 - Communication control characters – used in Transmission of text between remote devices – start of text, end of text, etc.

EBCDIC:

- EBCDIC stands for Extended Binary Coded Decimal Information Code.
- It is especially used in IBM computers.
- It is 8 bit code with 256 possible symbols.

Hollerith Code:

- It is a 12 bit code.
- Used in punched card readers of large computers.

ASCII Code (Decimal)

0 nul	16 dle	32 sp	48 0	64 @	80 P	96 `	112 p
1 soh	17 dc1	33 !	49 1	65 A	81 Q	97 a	113 q
2 stx	18 dc2	34 "	50 2	66 B	82 R	98 b	114 r
3 etx	19 dc3	35 #	51 3	67 C	83 S	99 c	115 s
4 eot	20 dc4	36 \$	52 4	68 D	84 T	100 d	116 t
5 enq	21 nak	37 %	53 5	69 E	85 U	101 e	117 u
6 ack	22 syn	38 &	54 6	70 F	86 V	102 f	118 v
7 bel	23 etb	39 '	55 7	71 G	87 W	103 g	119 w
8 bs	24 can	40 (	56 8	72 H	88 X	104 h	120 x
9 ht	25 em	41)	57 9	73 I	89 Y	105 i	121 y
10 nl	26 sub	42 *	58 :	74 J	90 Z	106 j	122 z
11 vt	27 esc	43 +	59 ;	75 K	91 [	107 k	123 {
12 np	28 fs	44 ,	60 <	76 L	92 \	108 l	124
13 cr	29 gs	45 -	61 =	77 M	93]	109 m	125 }
14 so	30 rs	46 .	62 >	78 N	94 ^	110 n	126 ~
15 si	31 us	47 /	63 ?	79 o	95 _	111 o	127 del

ASCII Code (Hex)

00 nul	10 dle	20 sp	30 0	40 @	50 P	60 `	70 p
01 soh	11 dc1	21 !	31 1	41 A	51 Q	61 a	71 q
02 stx	12 dc2	22 "	32 2	42 B	52 R	62 b	72 r
03 etx	13 dc3	23 #	33 3	43 C	53 S	63 c	73 s
04 eot	14 dc4	24 \$	34 4	44 D	54 T	64 d	74 t
05 enq	15 nak	25 %	35 5	45 E	55 U	65 e	75 u
06 ack	16 syn	26 &	36 6	46 F	56 V	66 f	76 v
07 bel	17 etb	27 '	37 7	47 G	57 W	67 g	77 w
08 bs	18 can	28 (	38 8	48 H	58 X	68 h	78 x
09 ht	19 em	29)	39 9	49 I	59 Y	69 i	79 y
0a nl	1a sub	2a *	3a :	4a J	5a Z	6a j	7a z
0b vt	1b esc	2b +	3b ;	4b K	5b [	6b k	7b {
0c np	1c fs	2c ,	3c <	4c L	5c \	6c l	7c
0d cr	1d gs	2d -	3d =	4d M	5d]	6d m	7d }
0e so	1e rs	2e .	3e >	4e N	5e ^	6e n	7e ~
0f si	1f us	2f /	3f ?	4f o	5f _	6f o	7f del

Represent ECE in ASCII code by using even parity.

Given Symbols	E	C	E
ASCII code in decimal	69	67	69
8-bit ASCII code with even parity	11000101	11000011	11000101

7-Segment Display.

The *7-segment display*, also written as “seven segment display”, consists of seven LEDs (hence its name) arranged in a rectangular fashion as shown. Each of the seven LEDs is called a segment because when illuminated the segment forms part of a numerical digit (both Decimal and Hex) to be displayed. An additional 8th LED is sometimes used within the same package thus allowing the indication of a decimal point, (DP) when two or more 7-segment displays are connected together to display numbers greater than ten.

Each one of the seven LEDs in the display is given a positional segment with one of its connection pins being brought straight out of the rectangular plastic package. These individually LED pins are labelled from a through to g representing each individual LED. The other LED pins are connected together and wired to form a common pin.

So by forward biasing the appropriate pins of the LED segments in a particular order, some segments will be light and others will be dark allowing the desired character pattern of the number to be generated on the display. This then allows us to display each of the ten decimal digits 0 through to 9 on the same 7-segment display.

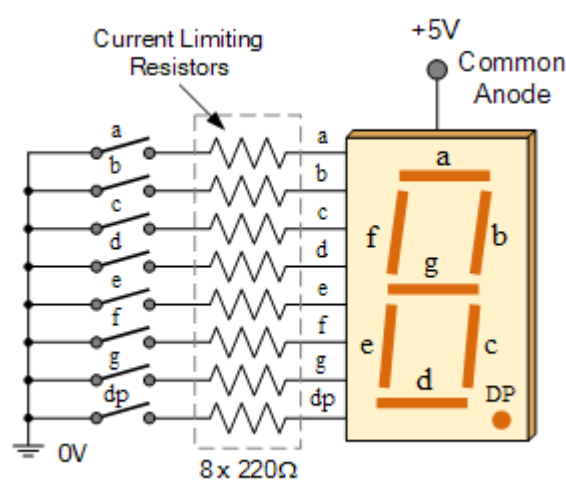


Fig. 2.29: Seven segment display circuit

16. Prescribed Text Books & Reference Books

Text Books:

1. M. Morris Mano, M.D. Ciletti, "Digital Design", 5th edition, Pearson, 2018

References:

1. Donald P Leach, Albert Paul Malvino, Goutam Saha, "Digital Principles and applications", Mc Graw Hill, 8th Edition, 2015.
2. David J. Comer, "Digital Logic & State Machine Design", Oxford University Press, 3rd Reprinted Indian Edition, 2012
3. R.D. Sudhakar Samuel, "Digital Logic Design", Elsevier Publishers.

17. Mini Project Suggestion

1. Traffic light control
2. Smart Notice Board using Seven Segment Display

